

PAL API Reference

Generated from public Protocol declarations and top-level factory signatures in `NativeCore.py`. Regenerate with `python Documentation/generators/generate_api_reference.py`; use `--check` to detect drift. These declarations describe the annotation/autocomplete surface; consult API Guide for behavior and the implementation/tests for runtime semantics.

Factories and utilities

```
FastMode()
NewIList()
MooreHood(dim, excludeCenter=False)
VonNeumannHood(dim, excludeCenter=False)
CircleHood(dim, rad, excludeCenter=False)
NewMultinomial(other=None)
NewGrid(dimensions, dtype)
NewAgentGrid(dimensions, numAgentProps=0, isStackable=False)
NewPopGrid(dimensions, capacity=None)
NewPDEgrid(dimensions)
```

Multinomial

Kind	Declaration
method	<code>Binomial(self, n: int, p: float) -> int</code>
method	<code>Setup(self, n: int)</code>
method	<code>Sample(self, p: float) -> int</code>

IList

Kind	Declaration
method	<code>__len__(self) -> int</code>
method	<code>__getitem__(self, index: int) -> int</code>
method	<code>Append(self, i: int)</code>
method	<code>Clear(self)</code>
method	<code>Random(self) -> int</code>
method	<code>Shuffle(self)</code>
method	<code>All(self) -> np.ndarray</code>
method	<code>Iter(self) -> np.ndarray</code>

AgentGrid

Kind	Declaration
method	<code>__len__(self) -> int</code>
property	<code>xDim(self) -> int</code>
property	<code>yDim(self) -> int</code>
property	<code>zDim(self) -> int</code>
property	<code>nDims(self) -> int</code>
property	<code>wrapX(self) -> bool</code>
property	<code>wrapY(self) -> bool</code>
property	<code>wrapZ(self) -> bool</code>
method	<code>Box(self, x1: int, x2: int, y1: int=None, y2: int=None, z1: int=None, z2: int=None)</code>
method	<code>Hood(self, hood, x: int, y: int=None, z: int=None, *, unroll: bool=False)</code>
method	<code>GetPop(self) -> int</code>
method	<code>Alive(self, agent: int) -> bool</code>
method	<code>NewAgentSQ(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>NewAgent(self, x: float, y: float=-1.0, z: float=-1.0) -> int</code>
method	<code>Dispose(self, agent: int)</code>
method	<code>I(self, agent: int) -> int</code>
method	<code>XSQ(self, agent: int) -> int</code>
method	<code>YSQ(self, agent: int) -> int</code>
method	<code>ZSQ(self, agent: int) -> int</code>
method	<code>X(self, agent: int) -> float</code>
method	<code>Y(self, agent: int) -> float</code>
method	<code>Z(self, agent: int) -> float</code>
method	<code>__getitem__(self, key) -> float</code>
method	<code>__setitem__(self, key, value: float)</code>
method	<code>ToI(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>ItoX(self, i: int) -> int</code>
method	<code>ItoY(self, i: int) -> int</code>
method	<code>ItoZ(self, i: int) -> int</code>
method	<code>InWrapSQX(self, value: int) -> int</code>
method	<code>InWrapSQY(self, value: int) -> int</code>

Kind	Declaration
method	<code>InWrapSQZ(self, value: int) -> int</code>
method	<code>InWrapX(self, value: float) -> float</code>
method	<code>InWrapY(self, value: float) -> float</code>
method	<code>InWrapZ(self, value: float) -> float</code>
method	<code>DispWrapX(self, x1: float, x2: float) -> float</code>
method	<code>DispWrapY(self, y1: float, y2: float) -> float</code>
method	<code>DispWrapZ(self, z1: float, z2: float) -> float</code>
property	<code>counts(self) -> object</code>
method	<code>LastAgent(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>MoveSQ(self, agent: int, x: int, y: int=-1, z: int=-1)</code>
method	<code>Move(self, agent: int, x: float, y: float=-1.0, z: float=-1.0)</code>
method	<code>AgentsAt(self, x: int, y: int=-1, z: int=-1)</code>
method	<code>AgentsInRadius(self, rad: float, x: float, y: float=-1.0, z: float=-1.0, *, exclude: int=None)</code>
method	<code>All(self, shuffle: bool=False) -> np.ndarray</code>

Grid

Kind	Declaration
method	<code>__len__(self) -> int</code>
method	<code>__getitem__(self, key)</code>
method	<code>__setitem__(self, key, value)</code>
property	<code>xDim(self) -> int</code>
property	<code>yDim(self) -> int</code>
property	<code>zDim(self) -> int</code>
property	<code>nDims(self) -> int</code>
property	<code>wrapX(self) -> bool</code>
property	<code>wrapY(self) -> bool</code>
property	<code>wrapZ(self) -> bool</code>
method	<code>Box(self, x1: int, x2: int, y1: int=None, y2: int=None, z1: int=None, z2: int=None)</code>
method	<code>Hood(self, hood, x: int, y: int=None, z: int=None, *, unroll: bool=False)</code>
method	<code>ToI(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>ItoX(self, i: int) -> int</code>
method	<code>ItoY(self, i: int) -> int</code>
method	<code>ItoZ(self, i: int) -> int</code>
method	<code>InWrapX(self, value: int) -> int</code>
method	<code>InWrapY(self, value: int) -> int</code>
method	<code>InWrapZ(self, value: int) -> int</code>

PopGrid

Kind	Declaration
method	<code>__len__(self) -> int</code>
property	<code>xDim(self) -> int</code>
property	<code>yDim(self) -> int</code>
property	<code>zDim(self) -> int</code>
property	<code>nDims(self) -> int</code>
property	<code>wrapX(self) -> bool</code>
property	<code>wrapY(self) -> bool</code>
property	<code>wrapZ(self) -> bool</code>
method	<code>Box(self, x1: int, x2: int, y1: int=None, y2: int=None, z1: int=None, z2: int=None)</code>
method	<code>Hood(self, hood, x: int, y: int=None, z: int=None, *, unroll: bool=False)</code>
method	<code>ToI(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>ItoX(self, i: int) -> int</code>
method	<code>ItoY(self, i: int) -> int</code>
method	<code>ItoZ(self, i: int) -> int</code>
method	<code>InWrapX(self, value: int) -> int</code>
method	<code>InWrapY(self, value: int) -> int</code>
method	<code>InWrapZ(self, value: int) -> int</code>
method	<code>__getitem__(self, key) -> int</code>
method	<code>__setitem__(self, key, value: int)</code>
method	<code>Add(self, value: int, x: int, y: int=-1, z: int=-1)</code>
method	<code>Update(self)</code>
method	<code>Reset(self)</code>
method	<code>GetPop(self) -> int</code>
method	<code>All(self) -> np.ndarray</code>

PDEgrid

Kind	Declaration
method	<code>__len__(self) -> int</code>
property	<code>xDim(self) -> int</code>
property	<code>yDim(self) -> int</code>
property	<code>zDim(self) -> int</code>
property	<code>nDims(self) -> int</code>
property	<code>wrapX(self) -> bool</code>
property	<code>wrapY(self) -> bool</code>
property	<code>wrapZ(self) -> bool</code>
method	<code>Box(self, x1: int, x2: int, y1: int=None, y2: int=None, z1: int=None, z2: int=None)</code>
method	<code>Hood(self, hood, x: int, y: int=None, z: int=None, *, unroll: bool=False)</code>
method	<code>ToI(self, x: int, y: int=-1, z: int=-1) -> int</code>
method	<code>ItoX(self, i: int) -> int</code>
method	<code>ItoY(self, i: int) -> int</code>
method	<code>ItoZ(self, i: int) -> int</code>
method	<code>InWrapX(self, value: int) -> int</code>
method	<code>InWrapY(self, value: int) -> int</code>
method	<code>InWrapZ(self, value: int) -> int</code>
method	<code>__getitem__(self, key) -> float</code>
method	<code>__setitem__(self, key, value: float)</code>
method	<code>Add(self, value: float, x: int, y: int=-1, z: int=-1)</code>
method	<code>Update(self)</code>
method	<code>Reset(self)</code>
method	<code>SetTimeSpaceStep(self, dt: float, dx: float, dy: float=1.0, dz: float=1.0)</code>
method	<code>Dx(self) -> float</code>
method	<code>Dy(self) -> float</code>
method	<code>Dz(self) -> float</code>
method	<code>Dt(self) -> float</code>
method	<code>Diffusion(self, rateConstant: float, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>DiffusionMask(self, rateConstant: float, mask=None)</code>

Kind	Declaration
method	<code>DiffusionField(self, rateConstants, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>DiffusionInterfaces(self, rateConstantsX, rateConstantsY=None, rateConstantsZ=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>DiffusionADI(self, rateConstant: float, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>DiffusionRadialCircle(self, rateConstant: float, outerBC=None)</code>
method	<code>DiffusionRadialSphere(self, rateConstant: float, outerBC=None)</code>
method	<code>Advection(self, vx: float, vy: float=0.0, vz: float=0.0, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>AdvectionField(self, xVels, yVels=None, zVels=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>
method	<code>AdvectionInterfaces(self, xVels, yVels=None, zVels=None, xMinBC=None, xMaxBC=None, yMinBC=None, yMaxBC=None, zMinBC=None, zMaxBC=None)</code>

PixWindow visualization

`StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False) -> tuple[Pix, PixWindow]`

Kind	Declaration
method	<code>IsOpen(self) -> bool</code>
method	<code>Update(self)</code>
method	<code>Save(self, path: str, block=False)</code>
method	<code>StartGif(self, path: str, delay=100)</code>
method	<code>AddGifFrame(self, block=False)</code>
method	<code>StopGif(self)</code>
method	<code>Close(self)</code>

OpenGLWindow visualization

`StartOpenGLWindow(xDim, yDim, zDim=None, width=800, height=800, title='PAL', headless=False) -> tuple[OpenGLDraw, OpenGLWindow]`

Kind	Declaration
method	<code>Update(self)</code>

Kind	Declaration
method	<code>IsOpen(self) -> bool</code>
method	<code>StartGif(self, path, delay=100)</code>
method	<code>AddGifFrame(self, block=False, timeout=30)</code>
method	<code>StopGif(self, timeout=30)</code>
method	<code>Circle(self, rad, color, x, y, z=None)</code>
method	<code>Box(self, xLen, color, x, y, z=None, yLen=None, zLen=None)</code>
method	<code>BoxSQ(self, color, x, y, z=None)</code>
method	<code>Line(self, width, color, x1, y1, x2, y2, z1=None, z2=None)</code>
method	<code>Borders(self, width, color)</code>
method	<code>Camera(self, x, y, z, yaw=None, pitch=None)</code>
method	<code>Clear(self)</code>
method	<code>Background(self, color)</code>
method	<code>Save(self, path: str, block=False)</code>
method	<code>Close(self)</code>

The `OpenGLWindow` table follows its source Protocol, which also contains method implementations; repeated declarations are listed once. `OpenGLDraw` is a compiled primitive collector, while the returned window handles rendering and output. Consult the API Guide for which object to use in Python and compiled drawing code.

Pix drawing buffer

Kind	Declaration
method	<code>__setitem__(self, key, color)</code>
method	<code>Xdim(self)</code>
method	<code>Ydim(self)</code>
method	<code>ToI(self, x, y)</code>
method	<code>ItoX(self, i)</code>
method	<code>ItoY(self, i)</code>
method	<code>__len__(self)</code>

OpenGLDraw primitive collector

Kind	Declaration
method	<code>Clear(self)</code>
method	<code>Circle(self, rad, color, x, y, z=0.0)</code>
method	<code>Box(self, xLen, color, x, y, z=0.0, yLen=-1.0, zLen=-1.0)</code>
method	<code>BoxSQ(self, color, x, y, z=0.0)</code>
method	<code>Line(self, width, color, x1, y1, x2, y2, z1=0.0, z2=0.0)</code>

Scope and validation

This reference lists public Protocol signatures, not internal safe/fast jitclass methods. A signature does not capture every runtime overload, indexing form, or validation rule. See the API Guide and executable tests. Visualization constructors and window Protocol methods are included; Pix buffer and OpenGLDraw method signatures are also included; consult the API Guide for behavior.